

SPECREGISTRY v1.0.0

The Control Plane for Spec-Driven AI

Versioning, Governing, and Observing the Context that Drives Code Generation

🔗 **The New Reality:** AI agents are generating code, but they need active constraints.

📁 **SpecRegistry:** A structured, verifiable SDD control plane.



SPEAKER PROMPT: SLIDE 1 OVERVIEW



"Hello everyone. Today, we're addressing a massive shift. As AI agents and human developers co-author code, we need explicit governance. SpecRegistry is our control plane to govern Markdown specs, deliver signed context, and verify that specs are economically optimized."

| Why Spec Driven Development?

ASPECT	TRIBAL KNOWLEDGE (STATUS QUO)	SPEC-DRIVEN DEVELOPMENT (SDD)
Source of Truth	Slack, stale READMEs, and fragmented reviews	Governed, version-controlled central Markdown specs
Agent Context	Injected ad-hoc, manual dumps, or guess behaviors	Scoped, signed, structured parameters via MCP Server
Review Gate	Post-hoc PR comments and unexpected rework	Specs verified & approved <i>*before*</i> writing code
Observability	None. Silent drift and unchecked rulesets	Continuous build drift checking and audit metrics



SPEAKER PROMPT: SLIDE 2 (SDD BENEFITS)

"Tribal knowledge doesn't scale with AI. AI models do not possess common sense or company memory. They are highly sensitive to context. SDD moves from reactive PR checking to proactive architectural alignment."

| The Cost of Poor Context

↻ Spec Drift

Codebase changes break documentation boundaries, forcing agents to operate on outdated criteria.

⊘ Spec Conflict

Contradictory instructions across docs trap reasoning agents in loops, failing implementations.

Token Saturation

Blindly dumping massive manuals into model prompts dilutes model attention and inflates costs.



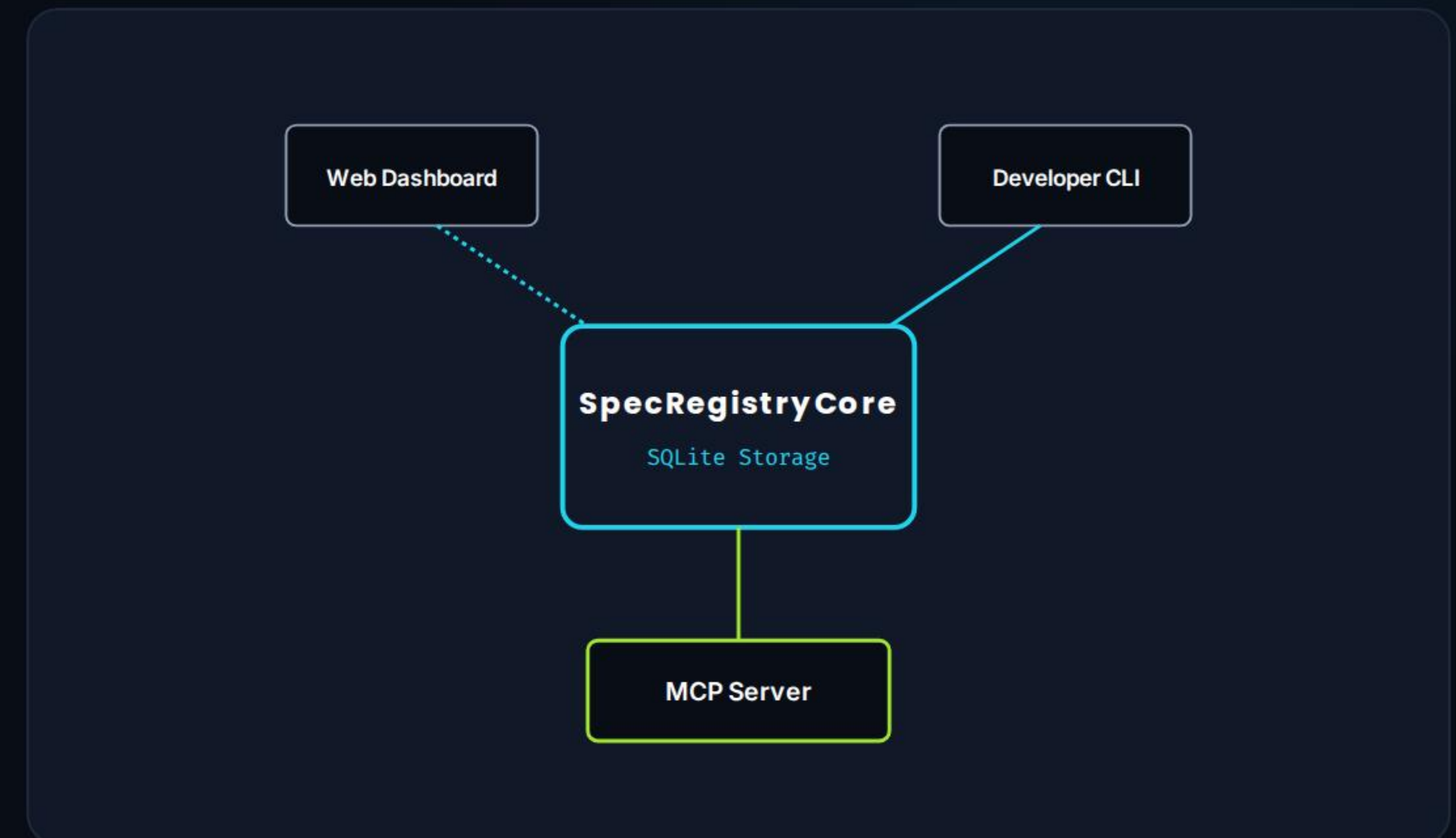
SPEAKER PROMPT: SLIDE 3 (COST ANALYSIS)



"If agents get stale or contradictory specs, they won't halt; they'll hallucinate. This loop details the consequences: high API costs, drift, and time-consuming developer rework."

| SpecRegistry Overview

-  **Centralized Hub:** Manages Global directives alongside localized project overrides.
-  **Semantic Versioning:** Specs progress through strict Change Requests and merge gates.
-  **Active Verification:** Checks heading layouts, dependencies, and conflicts deterministically.
-  **Context Compiler:** Compiles dynamic outputs matching IDE agent rules.



SPEAKER PROMPT: SLIDE 4 (ARCHITECTURE SUMMARY)

"SpecRegistry isn't a passive wiki; it's an active control plane. It coordinates design specs via semantic tracking, validating requirements before compilations push to local IDE rules."

| Three Architectural Pillars



Web Dashboard

Built with React & Vite. Offers a transparent review workspace to configure policies, approve specs, and analyze agent compliance reports.



Developer CLI

Node utility managing workspace configs, compiling specifications locally, and driving lint gates to block unapproved commits inside pipelines.



MCP Bridge Server

Provides dynamic search and targeted context lookup directly to agents, bypassing manual context copy-pasting loops entirely.



SPEAKER PROMPT: SLIDE 5 (THE 3 PILLARS)

"Let's detail our stack. Core storage relies on portable SQLite, API routines run under Fastify, CLI blocks local regression, and the Model Context Protocol feeds agents live instructions."

| Immutable Versioning



Phase 1: Draft

Proposal written as local Markdown layout. Configures semver schema v0.1.0.



Phase 2: Review

Lint passes evaluate head changes, auto-scanning specs for conflicts.



Phase 3: Approval

CODEOWNERS criteria evaluates and signs off. Locks configuration state.



Phase 4: Publish

Spec is committed and hashed, distributing context instantly to agents.



SPEAKER PROMPT: SLIDE 6 (THE PIPELINE)

"To prevent unapproved alterations, we enforce change request rules. Compatibility check APIs warn on MAJOR modifications, linting avoids discrepancies, and signatures enforce trust."

| The Active Feedback Loop

- 🔍 **Hybrid Querying:** Dynamic hybrid search provides precise specs, reducing token overhead.
- 🔗 **Stable Citations:** Injects stable section anchors so agents can trace rules back to mandates.
- ⚠️ **Active Reporting:** Agents trigger exceptions on vague or contradictory constraints.
- 🔧 **Automated Fixes:** Consolidates agent errors to assist human editors in drafting corrections.



SPEAKER PROMPT: SLIDE 7 (AGENT LOOPS)

"Our feedback loop provides trace capability. Instead of guessing, agents use the MCP tool to query specific layouts. If a spec is outdated or vague, they flag it, generating human review cues directly on the dashboard."

| CLI & CI/CD Integration

> CLI Commands

```
$ specreg init
# Sets up local workspace manifests
$ specreg generate
# Audits files & drafts initial specs
$ specreg check
# Verifies local code against spec logs
$ specreg sync
# Pulls & compiles local context rules
```

</> CI/CD Drift Check

```
# .github/workflows/verify.yml
- uses: specreg-check@main
  with:
    server: https://specs.corp.internal
    token: ${ secrets.REG_TOKEN }
    fail-on-drift: "true"
```

```
[DRIFT ALERT] Local DESIGN.md differs from registry. Build Blocked.
```



SPEAKER PROMPT: SLIDE 8 (DEV WORKFLOW)

"The developer workflow maps clean boundaries. Local sync compiles context rules, and `specreg check` blocks uncommitted modifications during pipeline executions, keeping central files intact."

| Tokenomics & Efficacy

Context Budgeting

Context space is expensive. We treat context as a finite resource, requiring specs to actively earn prompt inclusion.

ROI Diagnostics

The core API tracks spec lookups against actual agent code outcomes, identifying useless documentation overhead.

SPECIFICATION ROI METRICS



SPEAKER PROMPT: SLIDE 9 (TOKENOMICS)

"Context shouldn't be wasted. Our system diagnostic runs parallel task tests with and without a specific spec file to assess code efficacy. Low performing rules are flagged for refactoring."

| Enterprise Security & Scale



Two-Way Git Sync

Automated repository integration hooks.
Editing design files in GitHub triggers
Change Evaluations on the registry.



Slack Notifications

Direct system feedback alerts. Securely
accept or deny structural spec additions
directly from inside your channels.



SSO & RBAC Control

Map existing LDAP and Active Directory
groups to developer workspace access,
assuring complete directory governance.



SPEAKER PROMPT: SLIDE 10 (ENTERPRISE SECURITY)

"To fit global enterprise workspaces, SpecRegistry supports LDAP directories, active webhook triggers, Slack interactive responses, and a fully auditable event database out of the box."

| Adopt SpecRegistry: The ROI

-40%

Token Footprint

Dynamic context assembly serves only matching sections, avoiding prompt inflation.

95%

Agent Task Success

Eliminating outdated requirements removes structural agent guesswork.

0%

Spec Drift Failure

CI/CD build limits block unapproved local overrides, keeping specs aligned.



SPEAKER PROMPT: SLIDE 11 (ROI SUMMARY)

"Our metrics deliver a compelling pitch: we optimize token consumption via section search, align agents with telemetry, and enforce absolute design accuracy through build drift checks."

| How We Get Started

-  **Phase 1: Deploy Control Plane** — Host SpecRegistry locally via lightweight Docker stacks within secured enterprise zones.
-  **Phase 2: Seed Core Directives** — Load existing development parameters, common models, and styling guidelines.
-  **Phase 3: Connect Pilot Repositories** — Integrate the developer CLI into initial repositories to build out workspace configurations.
-  **Phase 4: Observe & Refine** — Review dynamic agent failure metrics and optimize specification content.

SPEAKER PROMPT: SLIDE 12 (NEXT STEPS)



"Our roadmap is clear and straightforward. We start by hosting locally, seed our common policies, integrate a pilot team repository, and continuously polish specs based on agent reports."



Questions & Discussion

Let's Align on Our Initial SpecRegistry Pilot Project Workspace

THANK YOU FOR YOUR TIME.



SPEAKER PROMPT: Q&A CONCLUSION

"Thank you everyone. I'd now love to open the floor to discuss technical setups or strategize which team project we want to choose for our pilot rollout."